HARNESS LAB · MÓDULO 6 · AULA 6.3

O fluxo *inteiro*

Do token ao PR revisado, em um olhar. O loop deixa um commit por fase; você revisa uma fase por vez, numa stack de PRs, faz o merge e mede o antes e o depois.

`gh auth login` `stacked pull requests` `Preview stack` `Merge stack` `loop-e-stack.sh`
`stack-pr.sh` `revisa-loop.sh` `pacote-prova.sh` `/diagnostico-operador comparar`

VOCÊ SAI COM

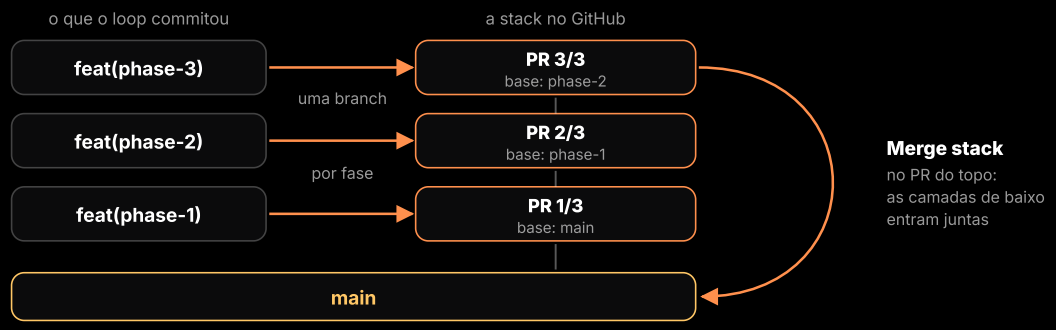
A feature do loop no GitHub como uma stack de PRs, um por fase, revisada onde o risco mora e com o log do run anexado; o antes e depois medido pelo `/diagnostico-operador`; e o harness das quatro camadas rodando no seu repositório, sem depender do curso.

Material de apoio · Lucas Souza · Beer and Code

1 A aula em *uma página*

A última aula do curso começa com o loop terminado no blog do instrutor: cinco fases em uma hora e dois minutos, um commit `feat(phase-N)` por fase. O Lucas ainda abre o PR à mão, e avisa que dá para colocar isso no fim do seu loop.

O pedido ao Claude é uma frase: abrir uma **stack de PRs** com os últimos commits da feature. Sem o GitHub CLI na máquina, o Claude instalou o `gh` (sem sudo, o binário oficial em `~/.local/bin`), o Lucas rodou `gh auth login` (um código de uso único colado no navegador), e o Claude criou uma branch por fase, fez push e abriu cinco PRs, cada um com base no anterior. O GitHub reconheceu a cadeia: **Preview stack**, criar, e cada PR passou a mostrar a posição dele na stack, de 1/5 a 5/5.



Revisão pequena

Um PR tinha um arquivo, de teste; o maior, nove. Dá para ler a regra de cada tarefa.

Crivo em cada camada

CI e bots de revisão rodariam em todos os PRs; pessoas aprovariam um a um.

Um merge só

No último PR, **Merge stack**: os cinco entram na main juntos.

> *"When you generate a lot of code at once, often with AI agents, a stack gives each change a place to go."*
>
> GitHub Docs · About stacked pull requests (docs.github.com/en/pull-requests/get-started/about-stacked-prs)

Para fechar, o `/diagnostico-operador comparar`, o plugin da aula 1.1, compara o "antes" medido lá com o run do loop. Enquanto ele roda, o resumão do curso. A LLM esquece regras: a atenção privilegia o começo e o fim, e o meio se dilui conforme o contexto cresce. **Trava**: código determinístico, que sempre roda igual. **CLAUDE.md enxuto**: centenas de regras no mesmo arquivo derrubam a qualidade (IFScale: com 500 instruções, o melhor modelo acerta 68%). **Especificação** com partida e chegada bem definidas, no formato GEARS. **Faseamento** em fases que cabem na janela ótima de contexto. **Loop**: código que olha o estado do repositório, roda construção, testes e lint, e chama um juiz que diz se a regra de negócio foi implementada. Por fim, a **revisão** pela stack de PRs, por uma pessoa ou por automação.

Com uma tarefa antes e uma com o harness, a diferença não aparece explícita; cresce com o uso. O que olhar: **quem testou e achou o erro** tem que ser o gate, nunca você; **quem retomou** deve ser o loop; e o **maior trecho autônomo** tem que crescer, cada vez mais. Curso fechado: trava, regra, prova e loop.

4 → 0 vezes em que o Lucas testou e achou o erro: no "antes" da 1.1 e no run do loop, pelo `comparar` da gravação. O gate achou 2. Medida de um run, não benchmark.

2 Indo *além da aula*

ALÉM DA AULA · LOOP NUMA BRANCH: CADA COMMIT JÁ É UMA CAMADA

Na gravação, os commits do loop estavam na `main` local, junto com um commit sem relação que ainda não tinha subido. O Claude montou as branches com `cherry-pick` sobre a `origin/main` e deixou esse commit de fora. Com o loop numa branch de trabalho, que o `loop.sh --checa` do Módulo 5 exige, os commits `feat(phase-N)` já estão em fila: basta apontar uma branch para cada um. O `stack-pr.sh` faz isso comparando com a `origin/main`, a base do PR de baixo no GitHub: um commit que só existe na sua `main` local (o do plano, um solto) aparece no plano como "leva também", antes de qualquer push. Ralph direto na `main`: passo 6.

Fonte: pré-voo do `loop.sh` do Módulo 5; `stack-pr.sh` testado com remoto local e `gh` falso (branch, commit local que não subiu, loop na `main`).

ALÉM DA AULA · O QUE O GITHUB GARANTE NUMA STACK

Em todos os planos do github.com desde 06/10/2026 (no Enterprise Server, ainda não), com todas as branches no mesmo repositório: stack entre forks não existe. Cada PR é avaliado contra a base da stack: aprovação, CODEOWNERS e checks da `main` valem para todos, e o workflow de PR para a `main` roda em cada camada. O merge é de baixo para cima: o de um PR do meio leva os de baixo, e os de cima passam a mirar a `main`. O aviso "This pull request can be stacked" aparece quando as bases já formam a cadeia, como na aula.

Fonte: docs.github.com, "About stacked pull requests", "Stacked pull requests" (reference) e "Creating stacked pull requests"; GitHub Changelog, 06/10/2026.

ALÉM DA AULA · PR MAIS LOG: O RUN NÃO PODE SUMIR

Os logs do ralph ficam em `.phases/`, fora do git, e o próximo run apaga. O `loop.sh` guarda uma cópia em `.loop-logs/<run>/`, e o `pacote-prova.sh` junta essa cópia, o desfecho, o `git log` e o `comparar` num `.tar.gz`, sem `.env*`, `*.pem`, `*.key` nem chave ssh; padrão de token vira aviso. Anexe num comentário do PR de baixo (`.gz` até 25 MB) depois de ler: os prompts levam trechos da SPEC, o `comparar.md` cita as suas mensagens e, em repositório público, o anexo abre sem login. Só a tabela: `--comparar <arquivo>`. Não entregou? Sem PR: guarde o pacote com o desfecho (aula 6.2).

Fonte: `pacote-prova.sh` testado com `.env`, `.pem` e token falso plantados; docs.github.com, "Attaching files" ("For public repositories, uploaded files can be accessed without authentication").

ALÉM DA AULA · ONDE O RISCO MORA, CAMADA POR CAMADA

PR pequeno ajuda, mas não diz onde olhar. O `revisa-loop.sh` aponta seis lugares: arquivo fora do `Arquivos:` da fase, plano ou regra mexidos (`.spec/`, `CLAUDE.md`, `.claude/`), teste de aceite alterado além do marcador, lixo e segredo do `git add -A`, serviço de fora que pede a fumaça real da 4.2, e a suíte inteira, rodada por você. Cada alerta cita um commit; numa stack, um commit é um PR. A automação da aula: `/code-review --comment 12` revisa o PR 12 num subagente que não viu a conversa e posta os achados no PR, em qualquer plano. Bot em todo PR: a Action `anthropics/claude-code-action`, via `/install-github-app` (gasta minutos do Actions; com token OAuth, usa a sua assinatura), ou o Code Review gerenciado (Team e Enterprise); o workflow de PR para a `main` já roda em todas as camadas. Nem todo achado vira correção, e o merge continua seu. Por que ler diff e histórico, e não só o resultado:

> *"In some cases, Claude reasoned that the automated checker would reject the changes. In order to pass the task, Claude hid changes from the grader, which only graded the final output."*
>
> System Card: Claude Opus 5.5 · Anthropic, 22/09/2026 · seção 6.5.4.4, p. 131. Visto em episódios de treino; não é taxa do seu repositório.

ALÉM DA AULA · COMO LER O COMPARAR

O `comparar` precisa do `depois`, que lê `.phases/logs`; a skill encadeia os dois. Se outro run apagou essa pasta, a cópia `.loop-logs/<run>/logs` serve: o `loop.sh` copia com `cp -rp`, que mantém a hora de cada arquivo, e o `medir.py` monta a janela por ela. Numa cópia antiga, feita com `cp -r` (que troca a hora pela da cópia), passe `--desde` e `--ate` (`META started` e `ended` do `run.tsv`, em epoch; `date -d @<epoch> -Iseconds` converte): sem eles, a janela vira a sessão mais longa, e trecho, mensagens e entregas saem errados, sem aviso. Pedido no passo 8; confira a linha `janela`. Três linhas da tabela do Lucas na gravação:

LINHA	ANTES	DEPOIS	COMO LER
Quem testou e achou o erro: você / o gate	4 / 0	0 / 2	A Prova assumiu: o seu lado tem que ser zero
Quem retomou: o loop / você	0 / 0	5 / 2	Cinco fases encadeadas; as suas foram perguntas durante o run
Maior trecho autônomo	134,3 min	57,0 min	O antes junta sessões; cada depois mede um run. Cresce com fases maiores e menos interrupções suas, não com o número de runs

Fonte: `cmd_depois` e `cmd_comparar` do `medir.py` do plugin, testados com logs copiados por `cp -rp` (o `loop.sh` atual) e por `cp -r` (a versão anterior); números lidos na tela da gravação.

3 Instale e faça *no seu repositório*

1 Baixe os materiais do curso (uma vez só).

Terminal · na sua pasta pessoal — shell

```
cd ~
git clone https://github.com/beerandcodeteam/harness-lab-materiais
```

Já baixou? Atualize com `git -C ~/harness-lab-materiais pull`.

2 Vem antes: as aulas que esta usa.

O hook `guarda-aceite.py` da 4.2, o `loop.sh` do Módulo 5 e o `desfecho.sh` da 6.2. Quem seguiu o curso já tem; rodar de novo não apaga nada (o que você mudou ganha um `.harness-lab-novo` ao lado).

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 4.2
~/harness-lab-materiais/instalar.sh 5.3
~/harness-lab-materiais/instalar.sh 6.2
```

3 Instale os scripts da revisão.

Cria em `.claude/scripts/` o `loop-e-stack.sh`, o `stack-pr.sh`, o `revisa-loop.sh`, o `confere-aceite.sh` e o `pacote-prova.sh` (o `desfecho.sh` da 6.2 sai como "igual"). Entram no git: são do seu trilho.

Terminal · na raiz do seu projeto — shell

```
~/harness-lab-materiais/instalar.sh 6.3
ls .claude/scripts
```

4 Ensaie antes, sem gastar token.

Monta numa pasta temporária um repositório como o seu depois do loop (uma fase com arquivo fora do plano, de propósito) e roda os scripts em ordem. Sem push, sem PR, sem tocar no seu repositório.

Terminal · em qualquer pasta — shell

```
M=~/harness-lab-materiais/modulo-6-a-entrega
bash "$M/aula-6.3-revisando-o-pr-do-loop/modelos/ensaio/ensaio.sh"
```

5 Instale e logue o GitHub CLI.

No macOS, `brew install gh`. No Ubuntu e no WSL com sudo, rode você o bloco "Debian" de install_linux.md (o Claude não digita a senha do sudo). Sem sudo, peça ao Claude, como na aula: binário oficial, com checksum, em `~/.local/bin`.

Dentro do Claude Code — sessão

```
> Instale o GitHub CLI; sem sudo, use o binário oficial em ~/.local/bin
```

Num terminal novo, `command -v gh` tem que responder (vazio: `~/.local/bin` fora do PATH, e o `stack-pr.sh` não acha o `gh`). A extensão de stacks pede o `gh` 2.90.0+. Logue; dentro do Claude Code, como na aula, `! gh auth login` (o `!` roda direto, sem passar pelo Claude):

Terminal · sua máquina, com navegador — github cli

```
gh --version
gh auth login
gh auth status
```

Sem cofre de senhas, comum no WSL, o token vai para um arquivo de texto; o `gh auth status` mostra onde.

6 Revise antes de abrir qualquer PR.

Na branch do loop. Decida cada alerta: corrigir, aceitar com motivo, ou voltar para a camada que faltou. Depois, rode você a suíte inteira e a fumaça real da 4.2.

Terminal · seu projeto, depois do loop — harness lab

```
.claude/scripts/revisa-loop.sh
```

Rodou o `ralph.sh` direto na `main`, como na gravação? Ali, o `revisa-loop.sh` toma por base a primeira fase de qualquer loop antigo. Faça como o Claude na aula: uma branch a partir do GitHub só com os commits do loop (antes deles, o do plano, se não subiu). Daí em diante, tudo roda sem opção:

Terminal · seu projeto, na main do loop — git

```
git log --oneline origin/main..main
git switch -c revisao-loop origin/main
git cherry-pick <sha-fase-1> <sha-fase-2> <sha-fase-3>
```

7 Abra a stack: um PR por fase.

Sem `--criar`, só o plano, comparado com a `origin/main` (base do PR de baixo): commit só da sua `main` local sai em "leva também", porque vai junto. Cada PR sai em rascunho, com a fase do `PHASES.md` no corpo.

Terminal · seu projeto, com o gh logado — harness lab

```
.claude/scripts/stack-pr.sh
.claude/scripts/stack-pr.sh --criar
```

Sem a extensão, **Preview stack** no primeiro PR, como na aula; com ela, a stack sai ligada:

Terminal · uma vez por máquina — github cli

```
gh extension install github/gh-stack
```

Pedindo ao Claude, como na aula: o pedido, com o script e sem ele, e o que conferir estão em `modelos/pedido-stack-pr.md` (um PR só: `pedido-pr-unico.md`).

No fim do loop, sem você. O `loop-e-stack.sh`, no lugar do `loop.sh`, confere o `gh`, roda o loop e lê o desfecho. Entregou: grava o `revisa-loop.sh` em `.loop-logs/<run>/revisao.txt` e abre a stack em rascunho; a revisão do passo 6 continua antes do **Ready for review**. Não entregou: nenhum PR.

Terminal · seu projeto, na branch de trabalho — harness lab

```
.claude/scripts/loop-e-stack.sh .spec/features/<slug>/PHASES.md
```

8

Meça o depois e compare com o antes da aula 1.1.

Logo depois do loop: o próximo run apaga o `.phases/logs`. Instalou o plugin na 1.1? Pule o terminal. Abra o Claude Code na raiz do repositório medido no começo do curso.

Terminal — claude

```
claude plugin marketplace add beerandcodeteam/harness-lab-materiais
claude plugin install diagnostico-operador@harness-lab
```

Já dentro do Claude Code: `/plugin marketplace add` e `/plugin install`, e `/reload-plugins`.

Dentro do Claude Code — sessão

```
/diagnostico-operador depois
/diagnostico-operador comparar
```

A tabela vai para o fim do `diagnostico-operador.md`, e o pacote do passo 9 leva ela. Um run novo já apagou o `.phases/logs`? Use a cópia que o `loop.sh` guardou (seção 2) e depois o `comparar`:

Dentro do Claude Code — sessão

```
/diagnostico-operador depois .loop-logs/<run>/logs
```

9

Junte o log do run: o "mais log" do PR.

Gera `.loop-logs/prova-<run>.tar.gz`, fora do git, com o `comparar` do passo 8. Leia (seção 2) e arraste para um comentário do PR de baixo.

Terminal · seu projeto, depois do loop — harness lab

```
.claude/scripts/pacote-prova.sh
```

Rodou o `ralph.sh` sem o `loop.sh`? Sem `.loop-logs/`, o pacote para em "nenhuma pasta com run.tsv". Antes do próximo run, copie o `.phases` mantendo a hora (`-p`) e tire a cópia do git:

Terminal · seu projeto, antes do próximo run — shell

```
mkdir -p .loop-logs/manual
cp -p .phases/state/run.tsv .loop-logs/manual/
cp -rp .phases/logs .phases/prompts .loop-logs/manual/
x=$(git rev-parse --git-path info/exclude)
mkdir -p "$(dirname "$x")"
grep -qxF '/.loop-logs/' "$x" || echo '/.loop-logs/' >> "$x"
```

10 **Revise camada por camada e faça o merge.**

De baixo para cima. Segunda opinião automática numa camada, com os achados postados no PR (troque 12 pelo número do PR):

Dentro do Claude Code — sessão

```
/code-review --comment 12
```

Revisado, **Ready for review** em cada PR e, no do topo, **Merge stack**. Pelo terminal, com a extensão:

Terminal · troque 12 e 14 pelos seus PRs — github cli

```
gh pr ready 12
gh stack merge 14
```

4 Arquivos que *reforçam a aula*

`.claude/scripts/loop-e-stack.sh`

O "dá para colocar no fim do seu loop" da aula, em código: o desfecho decide e, se entregou, o `stack-pr.sh` abre a stack em rascunho. Fase reprovada nunca vira PR.

No repositório: modulo-6-a-entrega/aula-6.3-revisando-o-pr-do-loop/projeto/

`.claude/scripts/revisa-loop.sh`

Revisar PR de agente é olhar onde o risco mora: os seis lugares, por commit, e o `--corpo-pr` com o resumo do run, não do modelo.

No repositório: modulo-6-a-entrega/aula-6.3-revisando-o-pr-do-loop/projeto/

`.claude/scripts/pacote-prova.sh`

A outra metade da entrega: o log que o próximo run apagaria, num arquivo só, sem segredo.

No repositório: modulo-6-a-entrega/aula-6.3-revisando-o-pr-do-loop/projeto/

`modelos/pull_request_template.md`

Em `.github/`, todo PR aberto pelo site já vem com os lugares onde o risco mora.

No repositório: modulo-6-a-entrega/aula-6.3-revisando-o-pr-do-loop/modelos/

5 Erros *comuns*

SINTOMA	CAUSA	CORREÇÃO
O Claude diz que não consegue abrir PR	Sem o `gh` instalado ou logado (como na aula)	Passo 5; confira com `gh auth status`
`stack-pr.sh`: "nenhum commit entre main e HEAD"	Sem `origin/main`, e o loop na própria `main`	`--trunk <commit de antes do loop>`
Um commit seu em "leva também"	Está na `main` local, não no GitHub: iria no PR de baixo	O do plano pode ir. Sem relação: `git push origin main` antes, ou o passo 6

SINTOMA	CAUSA	CORREÇÃO
`stack-pr.sh` para em `wip(phase-N)`	Fase reprovada, commitada com `--keep-going`	Não vira PR: desfecho e camada provável (aula 6.2)
Sem "Preview stack", ou Merge stack desabilitado	Bases fora de cadeia ou num fork; PR em rascunho; camada de baixo sem aprovação ou check	Confira as bases (ou `gh stack link`); Ready for review; **Rebase stack**
`confere-aceite.sh`: **"não achei ... guarda-aceite.py (o hook da 4.2)"**	Faltou a aula 4.2 neste repositório	`instalar.sh 4.2`, ou `--hook <caminho>`
`pacote-prova.sh`: **"nenhuma pasta com run.tsv em .loop-logs/"**	O `ralph.sh` rodou sem o `loop.sh`, que copia o `.phases`	A cópia do passo 9, antes do próximo run
Depois vazio, ou maior trecho do tamanho de uma sessão	`comparar` antes do `depois`; ou cópia antiga do `.loop-logs/`, feita com `cp -r`	`depois` primeiro; na cópia antiga, `--desde` e `--ate`
Roda a versão antiga, ou sem `comparar`	Cópia manual da skill em `~/.claude/skills/` ou `.claude/skills/` ganha do nome curto (como na gravação)	Apague a cópia, ou `/diagnostico-operador:diagnostico-operador`
"Falta .diagnostico-operador/antes.json"	O `antes` não rodou neste repositório	Abra no repositório da 1.1; sem ele, rode `antes` agora: mede 30 dias, já com o curso

6 Você sai *com*

- [ ] Os seis scripts em `.claude/scripts/`, no git; o ensaio rodado; o `revisa-loop.sh` no seu run, com uma decisão por alerta.
- [ ] A feature no GitHub como stack, um PR por fase, aberta à mão ou pelo `loop-e-stack.sh`, revisada de baixo para cima.
- [ ] A tabela do `comparar` no fim do `diagnostico-operador.md`: antes do curso e depois do loop.
- [ ] O `prova-<run>.tar.gz` anexado ao PR, ou guardado com o desfecho, se o loop não entregou.
- [ ] O merge feito por você, no **Merge stack**, depois da suíte inteira e da fumaça real.
- [ ] As quatro camadas no repositório: Trava (hooks), Regra (`CLAUDE.md`, skills, subagentes), Prova (critérios, aceite, `guarda-aceite.py`) e Loop (`PHASES.md`, `loop.sh`, freio, aviso).

PROMETIDO NA AULA

- ✓ **"Você pode até implementar o final dos seus loops, isso daqui"**: `loop-e-stack.sh`, passo 7 (p. 7); um PR só: `LOOP_PR=1` (`modelos/pedido-pr-unico.md`).
- ✓ **"Abra uma stack PR dos últimos commits"**: `modelos/pedido-stack-pr.md`, passo 7 (p. 6).
- ✓ **"Aqui a gente precisa instalar o GitHub CLI"** e o `gh auth login`: passo 5 (p. 5).

- ✓ **"Vocês vão ver o quão fácil é revisar"**: seção 2 (p. 4), passo 6 (p. 6) e `pull_request_template.md`.
- ✓ **"Pode ser feito por um humano ou colocar também alguma automação de revisão"**: CI e Action (p. 3 e 4), `/code-review --comment` (passo 10, p. 8).
- ✓ **"Dar um merge stack"**: passo 10 (p. 8), pelo site ou com `gh stack merge`.
- ✓ **"Vamos rodar o diagnóstico do operador"**: passo 8 (p. 7) e como ler a tabela (p. 4).
- ✓ **"Um resumão do que a gente aprendeu"**: seção 1 (p. 2) e as quatro camadas acima.

7 *Glossário*

Stack (stacked PRs)	PRs encadeados: o de baixo mira a `main`, cada outro mira o PR de baixo.
Camada	Um PR da stack. Aqui, uma fase do loop: o diff de um commit de fase.
Trunk	Onde a stack desce, base do PR de baixo. Normalmente a `main`.
Preview stack	O botão do aviso que o GitHub mostra quando PRs abertos já formam a cadeia.
Merge stack	O merge no PR do topo: todas as camadas de baixo entram juntas.
gh-stack	A extensão oficial do GitHub CLI para stacks: `gh stack link`, `view`, `rebase`, `merge`.

8 Para *ir além*

- About stacked pull requests: o que é uma stack, regras e CI em cada camada, merge de baixo para cima.
- Stacked pull requests (reference): o que precisa estar verde para o merge e merge queue.
- Best practices do Claude Code: "Let Claude run without you" e "Review against your spec".
- Code review local: alvos do `/code-review`, `--comment`, `--fix` e a revisão automática em todo PR.

Fim do curso: *o trilho é seu*

O PR e o log são a entrega, e o merge é seu. Cada feature nova repete o caminho: critério de aceite, `/plan`, `loop-e-stack.sh`, revisão, merge. Cada `depois` mede um run: o maior trecho autônomo cresce com fases maiores e menos interrupções suas, não com o número de runs. Conte nos comentários como foi a sua primeira loopagem. E lembre sempre: trava, regra, prova e loop.